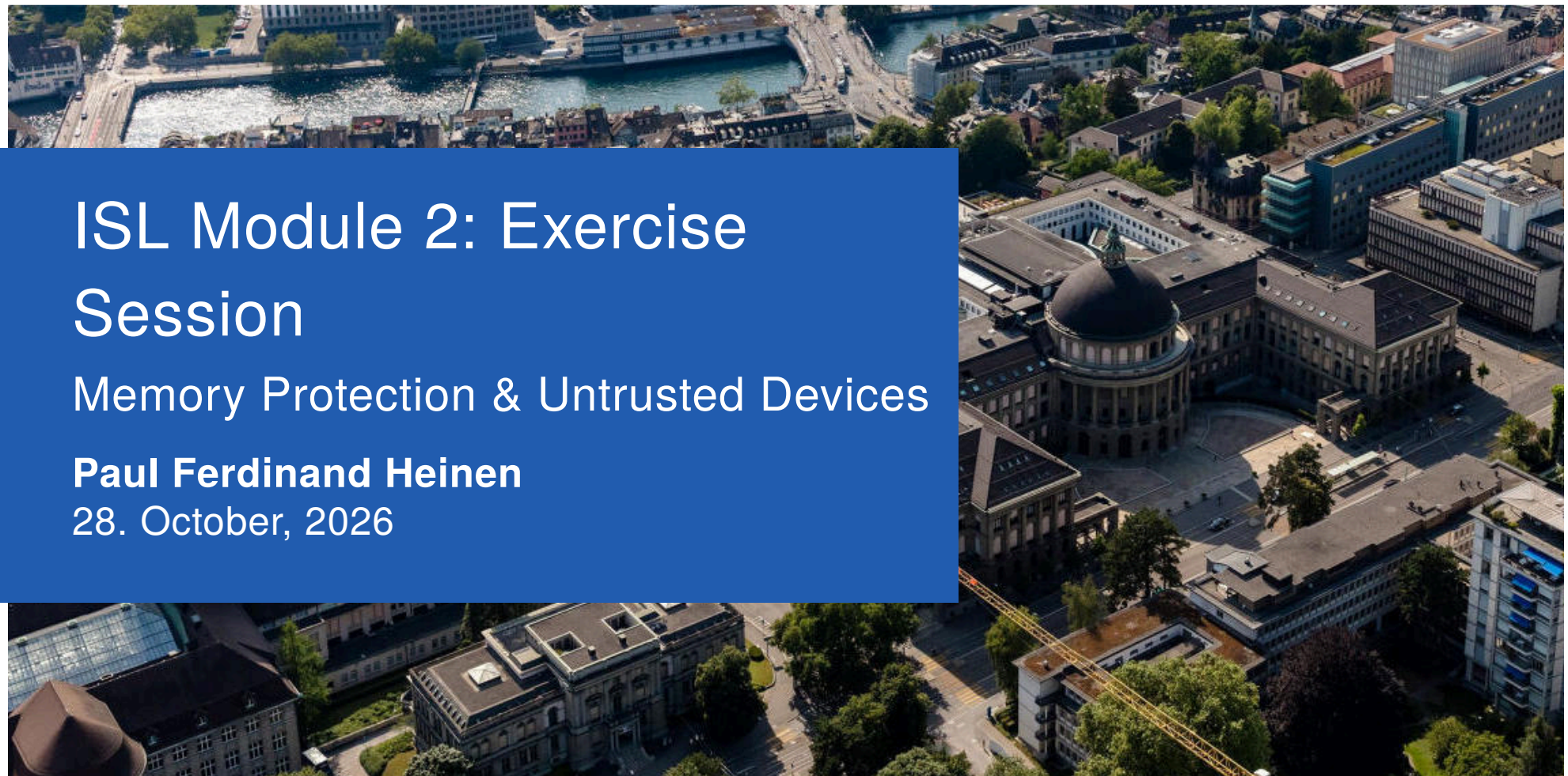


ISL Module 2: Exercise Session

Memory Protection & Untrusted Devices

Paul Ferdinand Heinen

28. October, 2026

Outline

1. Primer: Memory Protection
2. Exercise 1: Who Stops This Attack?
3. Primer: Untrusted DMA Devices
4. Exercise 3: DMA Attacks
5. Trusted I/O: Skipping the Bounce Buffer
6. Exercise 2: Heracles (If there is time)

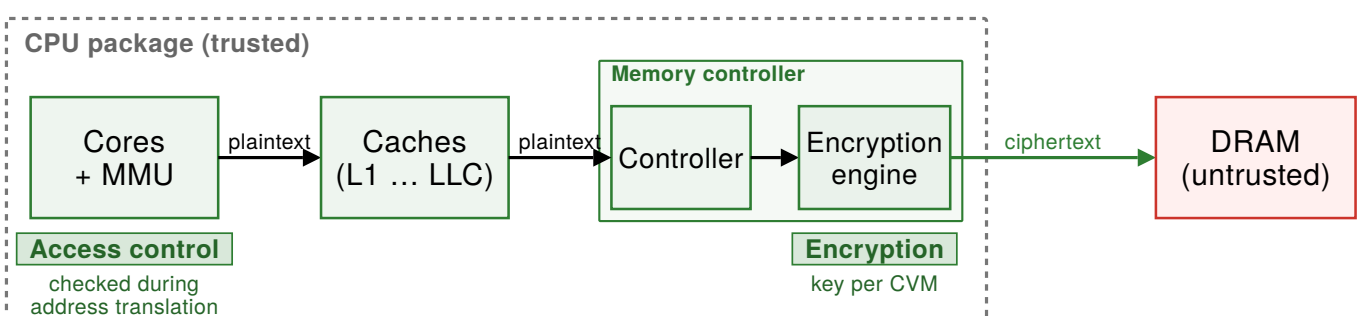
Two Ways to Protect Memory

Access control

- Hardware checks every access
- Typically done while translating addresses (MMU / IOMMU)
- Unauthorized access is blocked or faults

Cryptographic protection

- Encryption engine between the last trusted cache and DRAM
- Data leaves the CPU package only encrypted
- Even if the attacker reads memory, the data is useless



Access Control: Requirements

- **Coverage:** *all* paths that can reach the data must be checked
  - CPU accesses, but also device (DMA) accesses

Access Control: Requirements

- **Coverage:** *all* paths that can reach the data must be checked
  - CPU accesses, but also device (DMA) accesses
- **Configurator:** only trusted components may change the policy
  - The hypervisor must not rewrite it
  - A CVM must not edit it for other CVMs

Access Control: Requirements

- **Coverage:** *all* paths that can reach the data must be checked
  - CPU accesses, but also device (DMA) accesses
- **Configurator:** only trusted components may change the policy
  - The hypervisor must not rewrite it
  - A CVM must not edit it for other CVMs
- **Freshness:** a policy change must remove stale state
  - Flush TLBs (Translation Lookaside Buffers) that still hold the old permissions

What Exactly Needs to Be Checked?

- **Ownership:** which world / enclave / VM owns a physical page
  - Typically tracked at page granularity

What Exactly Needs to Be Checked?

- **Ownership:** which world / enclave / VM owns a physical page
  - Typically tracked at page granularity
- **Mapping integrity:** a malicious hypervisor cannot remap a protected page
  - The hypervisor still manages the mappings, but the TEE validates them

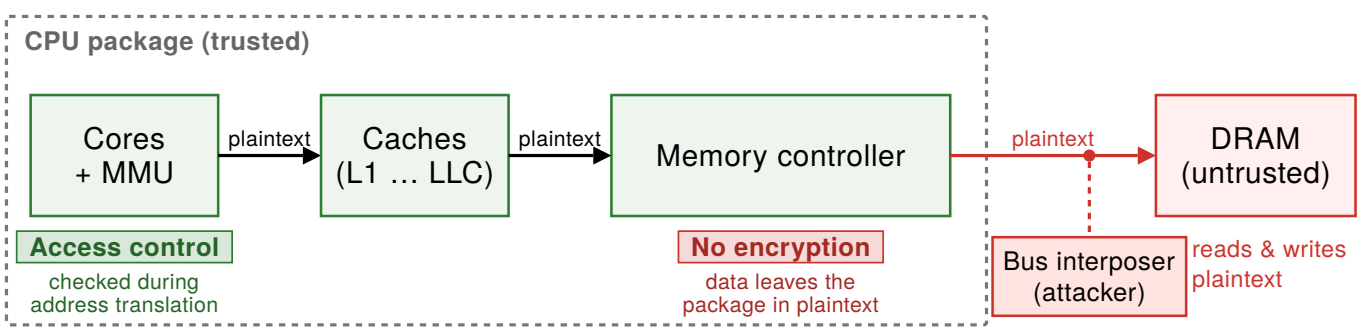
What Exactly Needs to Be Checked?

- **Ownership:** which world / enclave / VM owns a physical page
  - Typically tracked at page granularity
- **Mapping integrity:** a malicious hypervisor cannot remap a protected page
  - The hypervisor still manages the mappings, but the TEE validates them
- **Exclusivity:** a private page has exactly one owner

Why Cryptography Is Needed

Access control only works inside the CPU package.

- Data travels over the memory bus → an interposer can read it
- Some TEEs (e.g. SEV-SNP) do not check reads from the hypervisor
- A physical attacker can read/write to the bus or DRAM



What Must the Crypto Guarantee?

Property	Stops	Mechanism	Cost
Encryption	Passive reads (bus snooping, cold boot)	Encryption with address tweak (AES-XTS / XEX)	Low
Integrity	Modification (bus writes, Rowhammer)	MAC per cache line	Medium
Freshness	Replay of old ciphertext	Version counters + integrity tree (root on-chip)	High
Non-determinism	Ciphertext side channels	Randomness per write, or hide ciphertext	High

TEEs and Their Memory Protection

TEE	Access control	Encryption	Integrity guarantee through crypto	Freshness
SGX (client)	Yes (EPCM[1])	Yes	Yes (MAC)	Yes (integrity tree)
SGX (scalable)	Yes (EPCM)	Yes (AES-XTS)	No	No
AMD SEV-SNP	Yes (RMP[2])	Yes (AES-XEX)	No	No
Arm CCA	Yes (GPT[3])	Yes	No (optional)	No (optional)

[1] EPCM: Enclave Page Cache Map
[2] RMP: Reverse Map Table
[3] GPT: Granule Protection Table

TEEs and Their Memory Protection

TEE	Access control	Encryption	Integrity guarantee through crypto	Freshness
SGX (client)	Yes (EPCM[1])	Yes	Yes (MAC)	Yes (integrity tree)
SGX (scalable)	Yes (EPCM)	Yes (AES-XTS)	No	No
AMD SEV-SNP	Yes (RMP[2])	Yes (AES-XEX)	No	No
Arm CCA	Yes (GPT[3])	Yes	No (optional)	No (optional)

Take-away: most TEEs do not protect against active physical attackers, because tracking integrity and freshness is expensive.

[1] EPCM: Enclave Page Cache Map
[2] RMP: Reverse Map Table
[3] GPT: Granule Protection Table

Outline

1. Primer: Memory Protection

2. Exercise 1: Who Stops This Attack?

3. Primer: Untrusted DMA Devices

4. Exercise 3: DMA Attacks

5. Trusted I/O: Skipping the Bounce Buffer

6. Exercise 2: Heracles (If there is time)

Exercise 1: Task

For each scenario, decide:

- **Path:** how does the attacker reach the data? software · DMA · physical
- **Mechanism:** what must stop it? access control · encryption · integrity · freshness
- **Verdict:** does SGX (client) stop it? Does SEV-SNP?

We go through the six scenarios one by one.

After the second exercise session this week, the exercise slides and a detailed solution sheet to the exercises will be available in Moodle.

Scenario 1: Hypervisor Reads a Protected Page

The hypervisor maps an enclave/CVM page into its own address space and reads it.

Scenario 1: Hypervisor Reads a Protected Page

The hypervisor maps an enclave/CVM page into its own address space and reads it.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software	Access control / Encryption	**Blocked by access control**	**Read returns ciphertext only**

Scenario 1: Hypervisor Reads a Protected Page

The hypervisor maps an enclave/CVM page into its own address space and reads it.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software	Access control / Encryption	**Blocked by access control**	**Read returns ciphertext only**

- **SGX:** the EPCM blocks non-enclave access; reads return `0xFF`, writes are dropped
- **SEV-SNP:** the hypervisor reads ciphertext; the RMP blocks its writes

Scenario 2: Cold Boot

An admin freezes a DIMM and reads it in another machine.

Scenario 2: Cold Boot

An admin freezes a DIMM and reads it in another machine.

Path	Mechanism needed	SGX (client)	SEV-SNP
Physical	Encryption	**Protected**	**Protected**

Scenario 2: Cold Boot

An admin freezes a DIMM and reads it in another machine.

Path	Mechanism needed	SGX (client)	SEV-SNP
Physical	Encryption	**Protected**	**Protected**

- Memory only ever holds ciphertext; the key never leaves the CPU package
- Both TEEs encrypt memory, so a stolen DIMM is useless

Scenario 3: Malicious NIC

A malicious NIC writes via DMA into a protected page.

Scenario 3: Malicious NIC

A malicious NIC writes via DMA into a protected page.

Path	Mechanism needed	SGX (client)	SEV-SNP
DMA	Access control on the DMA path	**Blocked**	**Blocked**

Scenario 3: Malicious NIC

A malicious NIC writes via DMA into a protected page.

Path	Mechanism needed	SGX (client)	SEV-SNP
DMA	Access control on the DMA path	**Blocked**	**Blocked**

- **SGX:** the memory controller rejects DMA into the protected region (PRM)
- **SEV-SNP:** the IOMMU checks DMA against the RMP

Scenario 4: Replaying a Counter

A bus interposer records the ciphertext of a counter and writes it back later.

Scenario 4: Replaying a Counter

A bus interposer records the ciphertext of a counter and writes it back later.

Path	Mechanism needed	SGX (client)	SEV-SNP
Physical	Freshness	**Detected***	**Not detected**

Scenario 4: Replaying a Counter

A bus interposer records the ciphertext of a counter and writes it back later.

Path	Mechanism needed	SGX (client)	SEV-SNP
Physical	Freshness	**Detected***	**Not detected**

- **SGX (client):** the MEE checks a MAC and a tree of version counters → replay detected
- **SEV-SNP:** no MAC, no counters; old ciphertext at the same address decrypts cleanly
- * Scalable SGX dropped the integrity tree, so it does not detect this either

Scenario 5: Swapping Pages

The hypervisor swaps the physical pages behind two guest-physical addresses of a CVM.

Scenario 5: Swapping Pages

The hypervisor swaps the physical pages behind two guest-physical addresses of a CVM.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software	Mapping integrity	**Blocked by CPU**	**Detected via page validation**

Scenario 5: Swapping Pages

The hypervisor swaps the physical pages behind two guest-physical addresses of a CVM.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software	Mapping integrity	**Blocked by CPU**	**Detected via page validation**

- **SGX:** the EPCM stores each page's owner and virtual address; the CPU checks it on every TLB miss
- **SEV-SNP:** the RMP stores each page's guest-physical address; reassigning a page clears its "validated" bit → the guest notices

Scenario 6: Rowhammer

Code in a co-located VM hammers DRAM rows and flips a bit inside a protected page.

Scenario 6: Rowhammer

Code in a co-located VM hammers DRAM rows and flips a bit inside a protected page.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software → physical fault	Integrity	**Detected; machine locks**	**Silent corruption**

Scenario 6: Rowhammer

Code in a co-located VM hammers DRAM rows and flips a bit inside a protected page.

Path	Mechanism needed	SGX (client)	SEV-SNP
Software → physical fault	Integrity	**Detected; machine locks**	**Silent corruption**

- **SGX (client):** MAC mismatch on the next read → “drop and lock”: integrity kept, availability lost (SGX-Bomb)
- **SEV-SNP:** no integrity check; the flipped bit scrambles the whole 16-byte block, silently

Exercise 1: Summary

#	Path	Mechanism	SGX (client)	SEV-SNP
1	Software	Access control / Encryption	**Blocked**	**Ciphertext only**
2	Physical	Encryption	**Protected**	**Protected**
3	DMA	Access control (DMA)	**Blocked**	**Blocked**
4	Physical	Freshness	**Detected***	**Not detected**
5	Software	Mapping integrity	**Blocked**	**Detected**
6	Software → physical	Integrity	**Detected, machine locks**	**Silent corruption**

* Only SGX (client). Scalable SGX dropped the integrity tree.

Exercise 1: Summary

#	Path	Mechanism	SGX (client)	SEV-SNP
1	Software	Access control / Encryption	**Blocked**	**Ciphertext only**
2	Physical	Encryption	**Protected**	**Protected**
3	DMA	Access control (DMA)	**Blocked**	**Blocked**
4	Physical	Freshness	**Detected***	**Not detected**
5	Software	Mapping integrity	**Blocked**	**Detected**
6	Software → physical	Integrity	**Detected, machine locks**	**Silent corruption**

* Only SGX (client). Scalable SGX dropped the integrity tree.

Take-away: SEV-SNP exludes active physical attackers (no integrity, no freshness).

Outline

1. Primer: Memory Protection

2. Exercise 1: Who Stops This Attack?

3. Primer: Untrusted DMA Devices

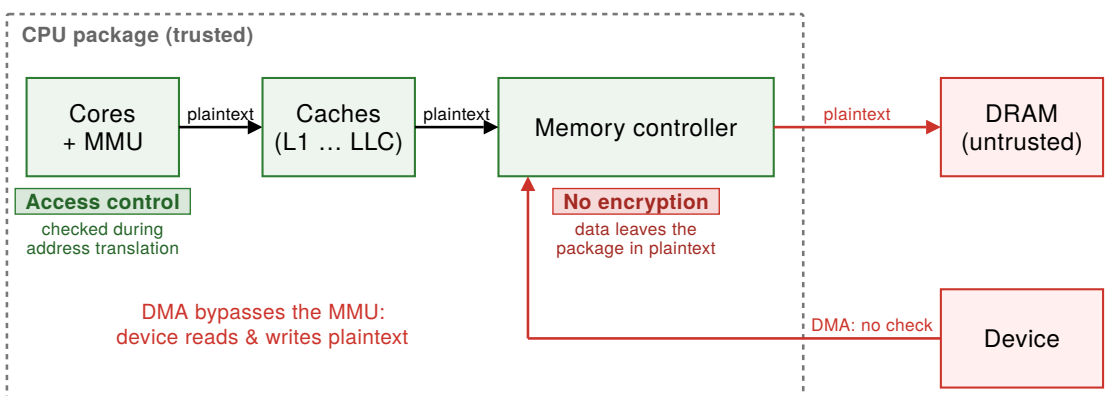
4. Exercise 3: DMA Attacks

5. Trusted I/O: Skipping the Bounce Buffer

6. Exercise 2: Heracles (If there is time)

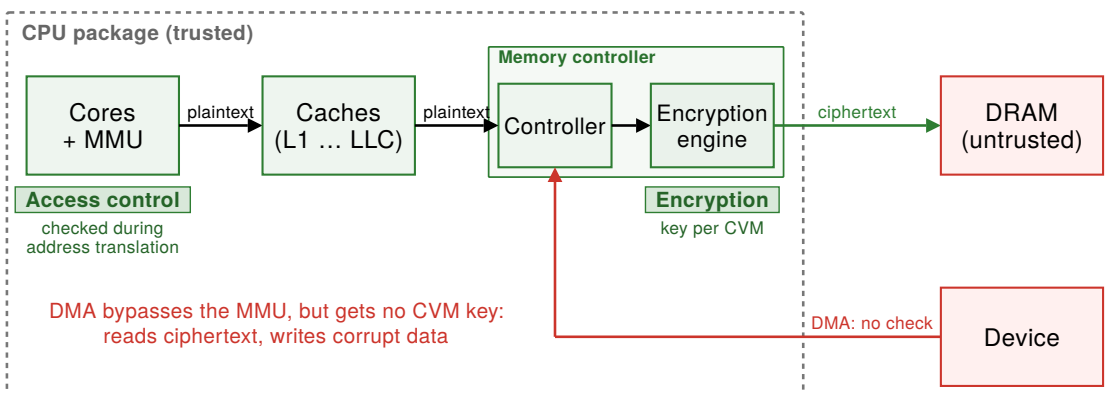
New Threat: Untrusted DMA Devices

- Many devices have Direct Memory Access (DMA): NICs, GPUs, storage, Thunderbolt peripherals
- DMA goes directly to memory, not through the CPU's MMU
- If DMA is not checked, a device has unrestricted access to all memory



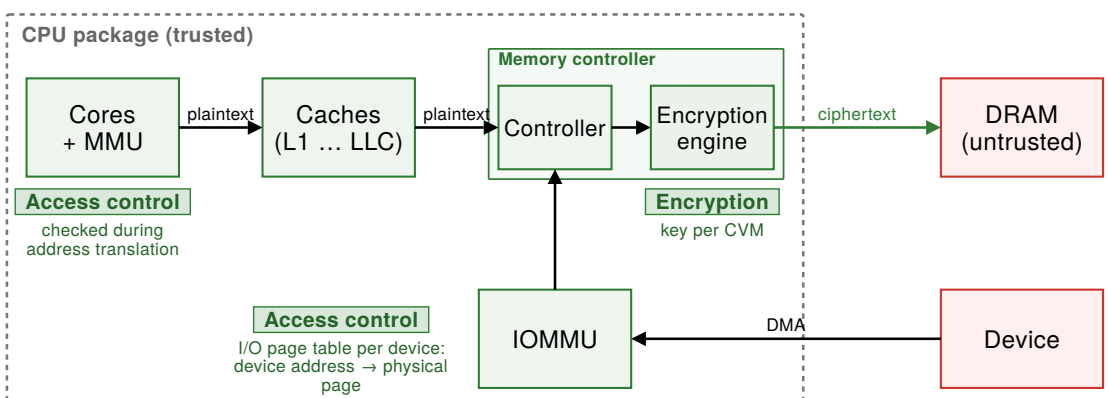
New Threat: Untrusted DMA Devices

- If DMA is not checked, a device can also access protected memory.
- However, it can only read or write ciphertext, because it does not have the encryption key.



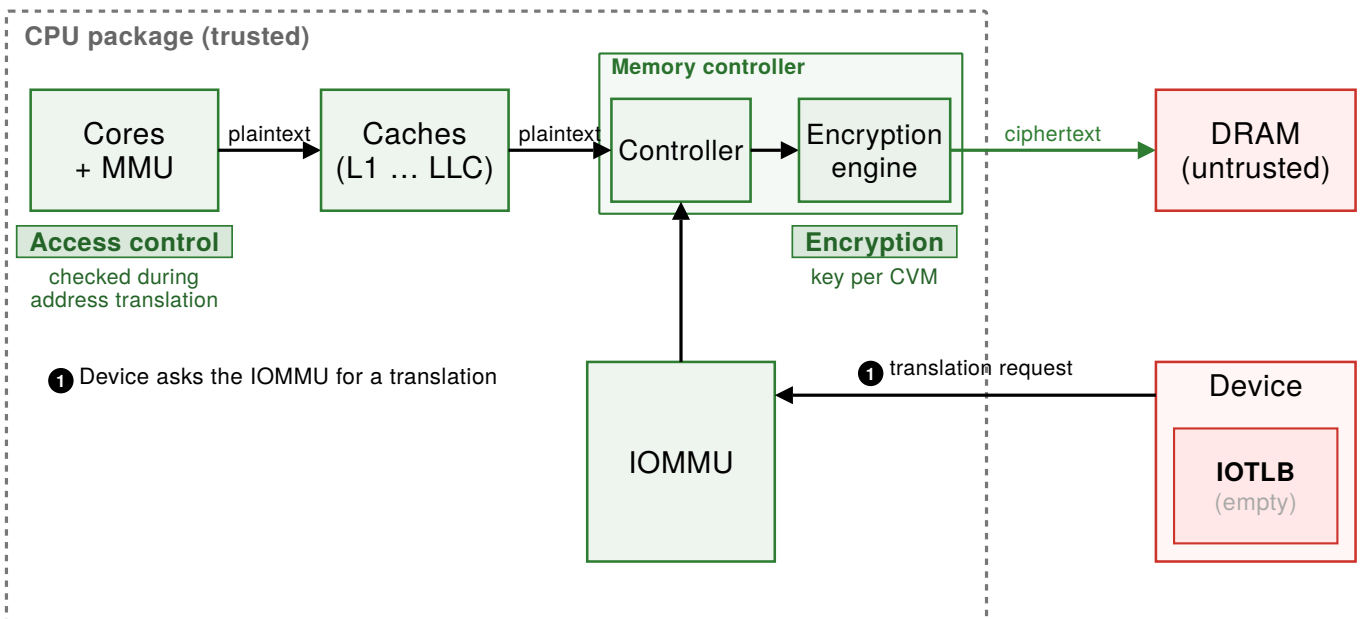
The IOMMU

- The IOMMU checks DMA like the MMU checks CPU accesses
- Per-device I/O page table: device address (IOVA) → physical page, at 4 KiB granularity



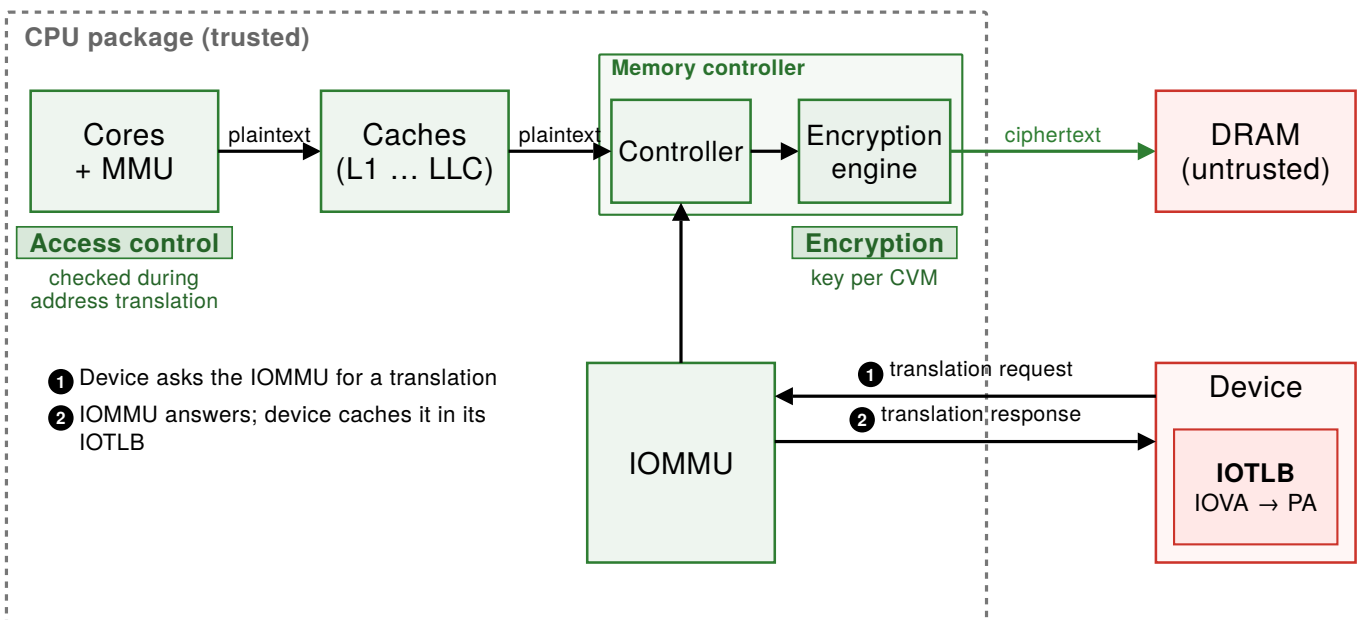
Address Translation Services (ATS)

- **ATS** (Address Translation Services): the device caches translations in its own IOTLB and sends “already translated” requests that the IOMMU does not re-check



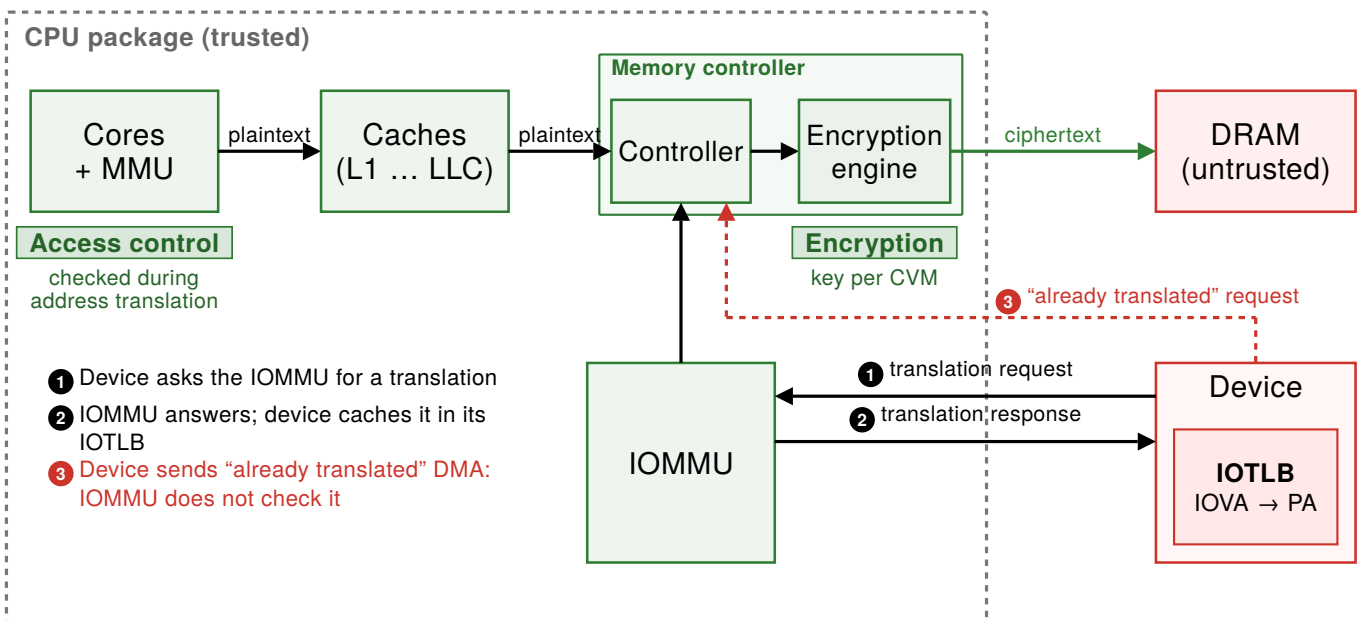
Address Translation Services (ATS)

- **ATS** (Address Translation Services): the device caches translations in its own IOTLB and sends “already translated” requests that the IOMMU does not re-check



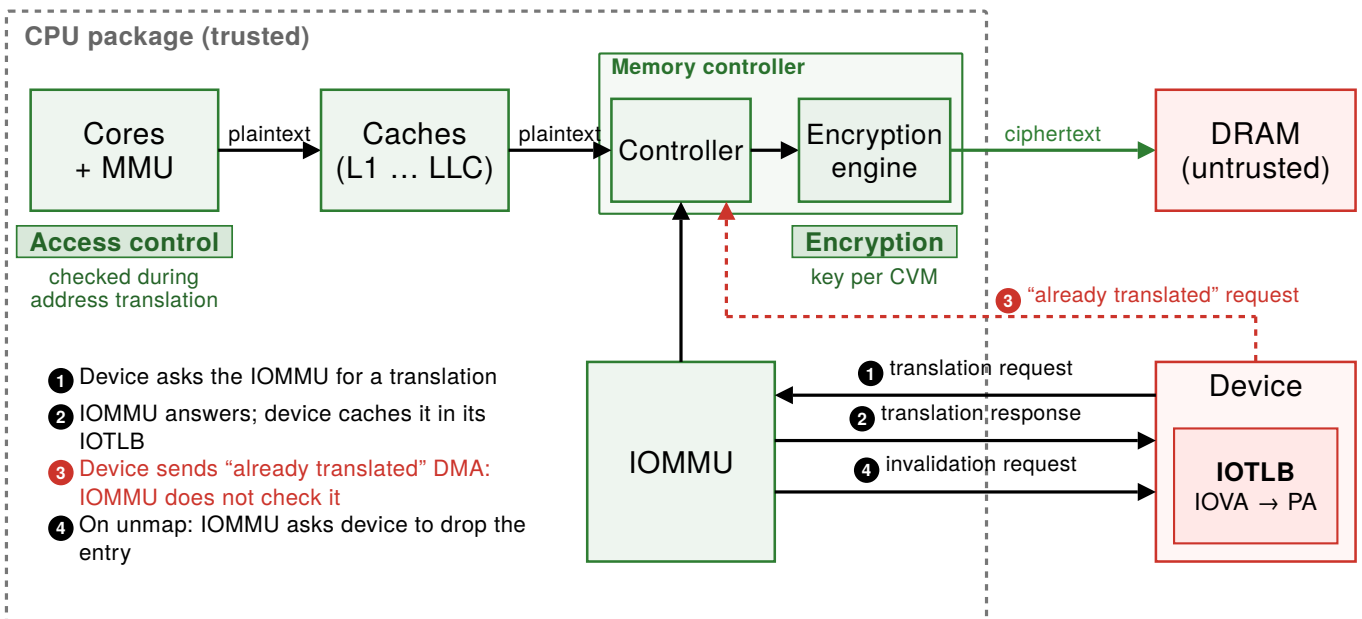
Address Translation Services (ATS)

- **ATS** (Address Translation Services): the device caches translations in its own IOTLB and sends “already translated” requests that the IOMMU does not re-check



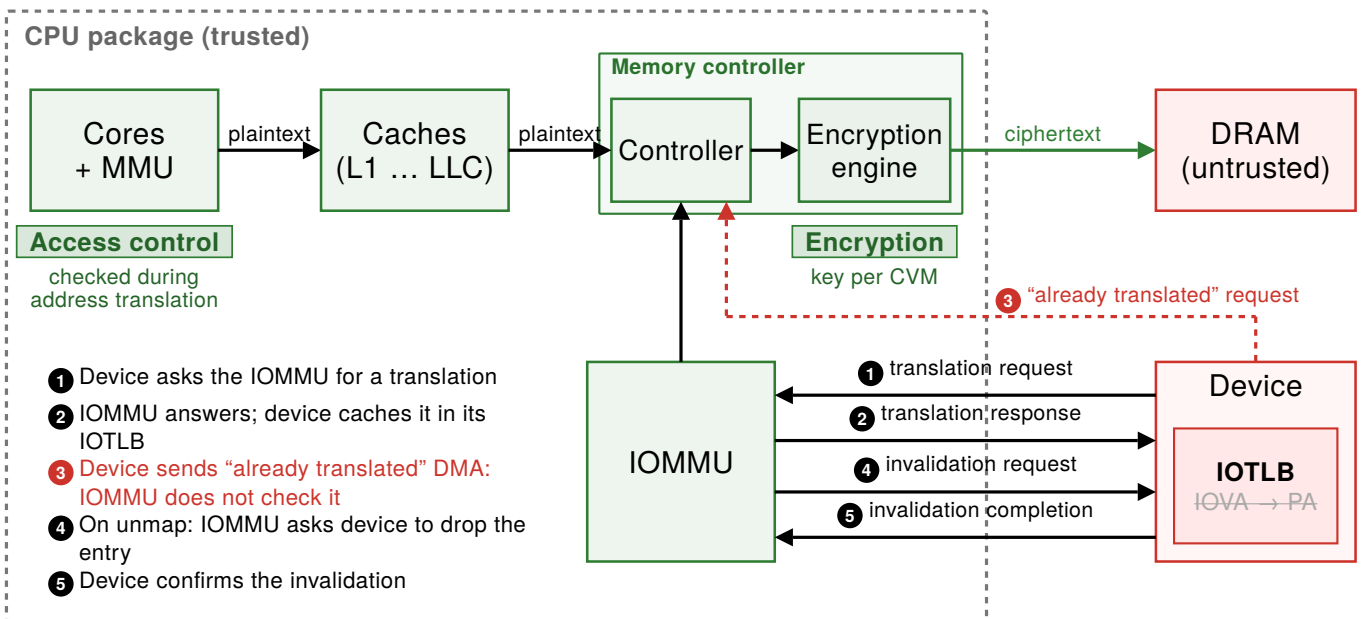
Address Translation Services (ATS)

- **ATS** (Address Translation Services): the device caches translations in its own IOTLB and sends “already translated” requests that the IOMMU does not re-check



Address Translation Services (ATS)

- **ATS** (Address Translation Services): the device caches translations in its own IOTLB and sends “already translated” requests that the IOMMU does not re-check



Outline

1. Primer: Memory Protection

2. Exercise 1: Who Stops This Attack?

3. Primer: Untrusted DMA Devices

4. Exercise 3: DMA Attacks

5. Trusted I/O: Skipping the Bounce Buffer

6. Exercise 2: Heracles (If there is time)

Exercise 3: Setting

- IOMMU with one I/O page table per device, read/write permissions
- **Attacker:** malicious PCIe device, sending arbitrary DMA request and ignoring protocol rules.[1]
- For now, we assume the hypervisor is trusted

[1] Based on: A. T. Markettos, C. Rothwell, B. F. Gutstein, A. Pearce, P. G. Neumann, S. W. Moore, R. N. M. Watson. “Thunderclap: Exploring Vulnerabilities in Operating System IOMMU Protection via DMA from Untrustworthy Peripherals.” NDSS 2019.

Task 1: ATS and the Device IOTLB

- The hypervisor enabled ATS because the device claims to support it
- The hypervisor frees a page P, previously used by the device, and reuses it
- The IOMMU immediately sends an invalidation request to the device when the page is freed

Task 1(a): Where Was the Translation?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(a) Where did a translation for P exist? Which of these places does the hypervisor control?

Task 1(a): Where Was the Translation?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(a) Where did a translation for P exist? Which of these places does the hypervisor control?

- In the I/O page table → controlled by the hypervisor, cleared on unmap
- In the device's IOTLB → lives on the device
- The hypervisor can only ask the device to clear its IOTLB (invalidation request)

Task 1(b): Has the Device Lost Access?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(b) The invalidation request was sent immediately. Is the device guaranteed to have lost access to P?

Task 1(b): Has the Device Lost Access?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(b) The invalidation request was sent immediately. Is the device guaranteed to have lost access to P?

- **No.** Invalidation needs the device to cooperate
- A malicious device answers but keeps using its stale translation
- P was reused → the device accesses whatever the kernel stores there now
- → Revocation is only as trustworthy as the device

Task 1(c): Is a Real Translation Needed?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(c) Does the device even need a real translation to perform an attack?

Task 1(c): Is a Real Translation Needed?

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(c) Does the device even need a real translation to perform an attack?

- **No.** The IOMMU does not check “already translated” requests
- The device can put any physical address into them
- → full IOMMU bypass

Task 1(d): Fixing the Attack

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(d) How would you fix the attack while keeping the device usable?

Task 1(d): Fixing the Attack

ATS is enabled because the device claims to support it. The hypervisor frees page P, which the device used before, and reuses it. The IOMMU immediately sends an invalidation request when the page is freed.

(d) How would you fix the attack while keeping the device usable?

- **Fix:** no ATS for untrusted devices; block pre-translated requests.
- → A device IOTLB is only safe if the device is trusted (e.g., attested)

Task 2: What About CVMs?

A CVM with a passed-through device and an untrusted hypervisor. Who configures the IOMMU and ATS? Can the CVM rely on it?

Task 2: What About CVMs?

A CVM with a passed-through device and an untrusted hypervisor. Who configures the IOMMU and ATS? Can the CVM rely on it?

- The untrusted hypervisor configures both. It can map CVM memory for any device or enable ATS → the IOMMU protects nothing for the CVM.

Outline

1. Primer: Memory Protection

2. Exercise 1: Who Stops This Attack?

3. Primer: Untrusted DMA Devices

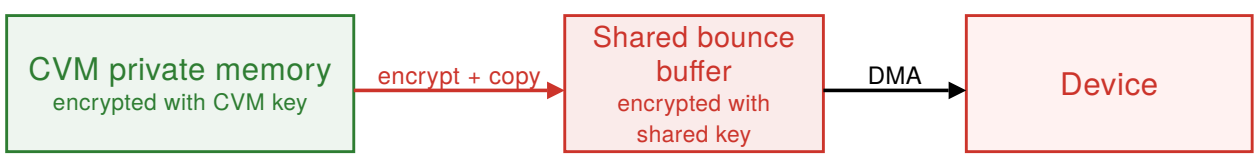
4. Exercise 3: DMA Attacks

5. Trusted I/O: Skipping the Bounce Buffer

6. Exercise 2: Heracles (If there is time)

The Bounce-Buffer Tax

- A passed-through device may only reach shared memory → every DMA goes through a bounce buffer
  - encrypt data with shared key in software and copy it into a shared page (for both directions)
- The copy and software encryption results in overhead, slowing done the communication path between the CVM and the device



The Goal: Direct Access to Private Memory

- We want an attested device to DMA directly into CVM private memory — no copy, no software crypto

The Goal: Direct Access to Private Memory

- We want an attested device to DMA directly into CVM private memory — no copy, no software crypto
- Problem: the device path is the IOMMU, and the IOMMU is programmed by the untrusted hypervisor
- Each path translates in two stages: stage-1 (guest: VA→IPA) and stage-2 (IPA→PA, hypervisor-controlled) — isolation lives in stage-2
- The TEE must verify stage-2 for the device (IOMMU), just as it already does for the CPU (MMU)

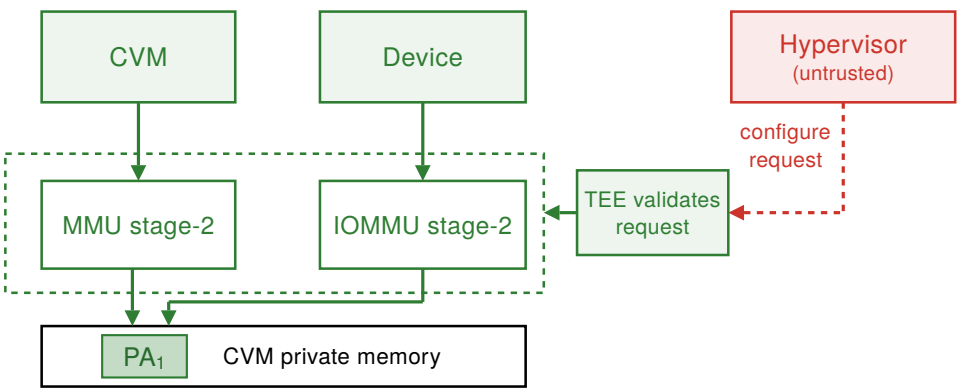
The Goal: Direct Access to Private Memory

- We want an attested device to DMA directly into CVM private memory — no copy, no software crypto
- Problem: the device path is the IOMMU, and the IOMMU is programmed by the untrusted hypervisor
- Each path translates in two stages: stage-1 (guest: VA→IPA) and stage-2 (IPA→PA, hypervisor-controlled) — isolation lives in stage-2
- The TEE must verify stage-2 for the device (IOMMU), just as it already does for the CPU (MMU)

Three things must then hold, enforced by the TEE:
1. **Isolation** — the device reaches only its CVM's memory
2. **Synchronized view** — device and CPU see the same physical page
3. **Exclusive, attested ownership** — a trusted device bound to one CVM

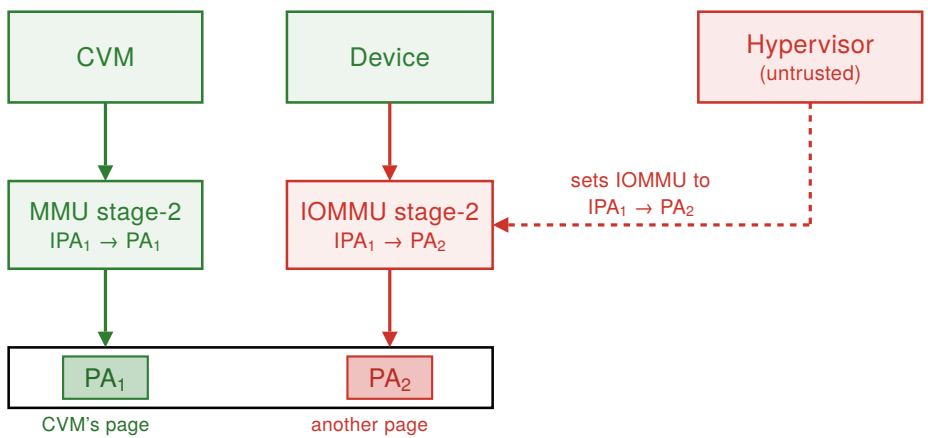
Requirement 1: Both Page Tables Under the TEE

- CPU accesses go through the MMU stage-2 tables, device accesses through the IOMMU stage-2 tables → the TEE must own check both
- The hypervisor still configures mappings, but the TEE validates every request before it applies → bad mappings are rejected



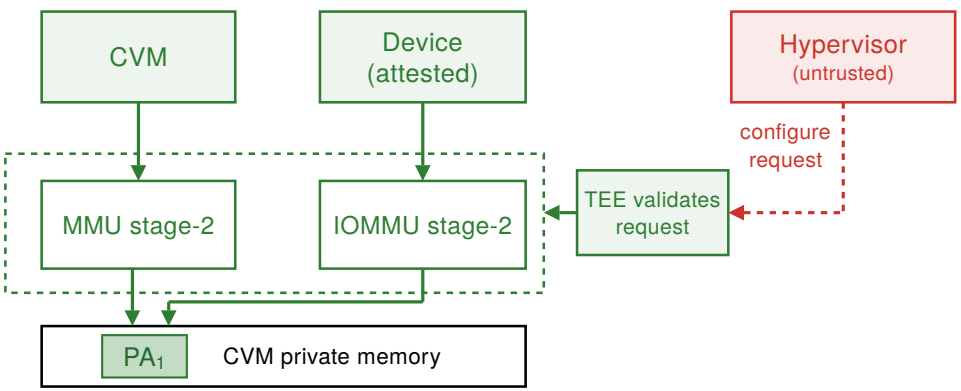
Requirement 2: Keep the Two Views Consistent

- Owning both tables is not enough — they must agree
- The MMU and IOMMU must use a one-to-one IPA→PA mapping, landing on the same physical page
- If the tables disagree — MMU: $IPA_1 \rightarrow PA_1$, IOMMU: $IPA_1 \rightarrow PA_2$ — the device's DMA is silently redirected to a page the CPU never sees



Requirement 3: Exclusive, Attested Ownership

- **Exclusive ownership:** a device belongs to exactly one CVM, checked in the TEE at VM creation
- **Attestation:** the device proves an unforgeable identity → it joins the CVM's trusted computing base
- **Encrypted PCIe link:** data leaving the package to the device stays encrypted (bus adversary)



Outline

1. Primer: Memory Protection

2. Exercise 1: Who Stops This Attack?

3. Primer: Untrusted DMA Devices

4. Exercise 3: DMA Attacks

5. Trusted I/O: Skipping the Bounce Buffer

6. Exercise 2: Heracles (If there is time)

Exercise 2: Setting

CVM memory is encrypted in blocks (4 bytes here, 16 in reality):[1]

$$C = E_K(P \oplus T(\text{addr})) \oplus T(\text{addr})$$

- E is deterministic, K is secret, the tweak T depends only on the address

The malicious hypervisor does not know K, but it can:

- **Read** the ciphertext of any CVM memory
- **Move** a CVM page to an address it chooses (re-encrypted with the new tweak, CVM does not notice)
- **Pause** the CVM at any point
- **Inject** chosen data into a CVM page (e.g., network packets)

[1] Based on: B. Schlüter, C. Wech, S. Shinde. “Heracles: Chosen Plaintext Attack on AMD SEV-SNP.” ACM CCS 2025.

Task 1: Why a Tweak?

Why does the tweak exist? What happens if the same value is written to the same address twice?

Task 1: Why a Tweak?

Why does the tweak exist? What happens if the same value is written to the same address twice?

- The tweak makes equal values at different addresses encrypt differently.
- At the same address, the ciphertext is identical: encryption is deterministic per address.

Task 2: Build a Test

A secret block S sits at address A. Build a test: *"Is S equal to my guess g?"*

Task 2: Build a Test

A secret block S sits at address A. Build a test: *"Is S equal to my guess g?"*

1. Snapshot the ciphertext at A
2. Move the victim page away from A
3. Inject g at the same offset in another page, move that page to A
4. Compare with the snapshot: match $\iff S = g$

→ a **chosen-plaintext oracle**

Task 3: How Many Guesses?

Worst case if S is completely unknown (4 bytes)? 16 bytes? Only one byte unknown?

Task 3: How Many Guesses?

Worst case if S is completely unknown (4 bytes)? 16 bytes? Only one byte unknown?

- 4 unknown bytes: 2^{32} guesses
- 16 unknown bytes: 2^{128}, infeasible
- 1 unknown byte: just 256

Task 3: How Many Guesses?

Worst case if S is completely unknown (4 bytes)? 16 bytes? Only one byte unknown?

- 4 unknown bytes: 2^{32} guesses
- 16 unknown bytes: 2^{128}, infeasible
- 1 unknown byte: just 256

→ The attacker wants secrets that change **one byte at a time** in an otherwise known block.

Task 4: Stealing a Password

Buffer at A starts as `00 00 00 00`, one character per keystroke, alphabet {a, e, s, t}. Each round: snapshot the block, then test every value for the new character.

Round	Snapshot	a	e	s	t
1	9E07C4A1	41D28F3B	7AB31E96	E610D25F	9E07C4A1
2	2C58A70E	D90F6B24	2C58A70E	B7A1093C	6E3CF581
3	F4135D8B	F4135D8B	0B9DE247	57E2B6A0	A86C3F19

How can the hypervisor recover the password? How many tries does it need in the worst case?

Task 4: Solution

Round	Snapshot	a	e	s	t
1	9E07C4A1	41D28F3B	7AB31E96	E610D25F	**9E07C4A1**
2	2C58A70E	D90F6B24	2C58A70E	B7A1093C	6E3CF581
3	F4135D8B	F4135D8B	0B9DE247	57E2B6A0	A86C3F19

Find the guess that matches the snapshot:

- **Round 1:**
  `[t 00 00 00]` → `t`

Task 4: Solution

Round	Snapshot	a	e	s	t
1	9E07C4A1	41D28F3B	7AB31E96	E610D25F	**9E07C4A1**
2	2C58A70E	D90F6B24	**2C58A70E**	B7A1093C	6E3CF581
3	F4135D8B	F4135D8B	0B9DE247	57E2B6A0	A86C3F19

Find the guess that matches the snapshot:

- **Round 1:**
  `[t 00 00 00]` → `t`
- **Round 2:**
  `[t e 00 00]` → `e`

Task 4: Solution

Round	Snapshot	a	e	s	t
1	9E07C4A1	41D28F3B	7AB31E96	E610D25F	**9E07C4A1**
2	2C58A70E	D90F6B24	**2C58A70E**	B7A1093C	6E3CF581
3	F4135D8B	**F4135D8B**	0B9DE247	57E2B6A0	A86C3F19

Find the guess that matches the snapshot:

- **Round 1:**
  `[t 00 00 00] → t`
- **Round 2:**
  `[t e 00 00] → e`
- **Round 3:**
  `[t e a 00] → a`

Password: **tea**

Task 4: Solution

How many tries does the hypervisor need in the worst-case to recover the password?

- Byte by byte: 3 rounds × 4 guesses = 12 guesses
- All at once: $4 \cdot 4 \cdot 4 = 4^3 = 64$ guesses

Task 4: Solution

How many tries does the hypervisor need in the worst-case to recover the password?

- Byte by byte: 3 rounds × 4 guesses = 12 guesses
- All at once: $4 \cdot 4 \cdot 4 = 4^3 = 64$ guesses

Real attack (16-byte block, 256 values per byte): $16 \cdot 256 = 4096$ guesses byte by byte, instead of $256^{16} = 2^{128}$ at once. Learning one byte at a time turns an exponential cost into a linear one.

Task 5: Missed Keystrokes & Defenses

What happens if the attacker misses a keystroke?
Name one defense that stops the attack.

Task 5: A Missed Keystroke

The attacker misses the `e` and only sees the block again after the `a`:

`[t 00 00 00]` → *missed* → `[t e a 00]`

Task 5: A Missed Keystroke

The attacker misses the `e` and only sees the block again after the `a`:

`[t 00 00 00]` → *missed* → `[t e a 00]`

Now two characters are unknown, so it must try every pair `[t ? ? 00]`:

	a	**e**	**s**	**t**
a	`[t a a 00]`	`[t a e 00]`	`[t a s 00]`	`[t a t 00]`
e	**`[t e a 00]`**	`[t e e 00]`	`[t e s 00]`	`[t e t 00]`
s	`[t s a 00]`	`[t s e 00]`	`[t s s 00]`	`[t s t 00]`
t	`[t t a 00]`	`[t t e 00]`	`[t t s 00]`	`[t t t 00]`

Task 5: The Cost of a Missed Write

- One missed keystroke: $4 \times 4 = 16$ guesses instead of 4
- Every missed keystroke multiplies the cost by 4: k missed $\rightarrow 4^{k+1}$ guesses
- The cost grows exponentially $\rightarrow$ the attacker must catch every write

Task 5: The Cost of a Missed Write

- One missed keystroke: $4 \times 4 = 16$ guesses instead of 4
- Every missed keystroke multiplies the cost by 4: k missed $\rightarrow 4^{k+1}$ guesses
- The cost grows exponentially $\rightarrow$ the attacker must catch every write

Real bytes (256 values): one missed byte $\rightarrow 256^2 = 65536$ guesses instead of 256.

Task 5: Defenses

Which defense stops the attack?

Task 5: Defenses

Which defense stops the attack?

- **Forbid the hypervisor from moving pages**
  AMD firmware update
- **Hide ciphertext from the hypervisor**

Task 5: Defenses

Which defense stops the attack?

- **Forbid the hypervisor from moving pages**
  AMD firmware update
- **Hide ciphertext from the hypervisor**

- A stronger AES key does not help: the attack never breaks AES, it only exploits the fact that encryption is deterministic per address.